EVENTSPHERE

Simple teammate guide

> **Goal:** help a frontend teammate understand what the system already does, how the parts connect, and what is safe to explain to a teacher.

This guide uses the current working implementation reviewed from the project's working branch. It uses normal language instead of technical implementation details.

1. What is EventSphere?

EventSphere is a college event management platform. It connects the main people and workflows involved in running a college event.

In simple words, it covers this journey:

Organizer creates event -> adds activities -> participant discovers an activity -> participant registers -> registration may need approval -> ticket/QR is used -> volunteer scans -> attendance is recorded.

Main users

User	Simple meaning	What they do
Organizer	Person running the event	Creates events, activities, manages registrations and volunteers
Participant	Student joining events	Browses, registers, creates/joins teams, uses tickets
Volunteer	Event support staff	Performs QR check-in for the assigned scope
Judge	Possible project role	Role exists, but judging/scoring workflow is not implemented yet

> **Teacher-friendly line:** "EventSphere connects event creation, activity registration, QR tickets, volunteer check-in and attendance in one platform."

2. Event and Activity

Event means the overall program. Example: *TechFest 2026*.

Activity means one participant-facing item in that event. Example: Hackathon, Quiz, Workshop.

Why are they separate?

Because different activities can have different rules. A hackathon can allow teams of 2-4 people, while a quiz may allow only individual registration. Capacity, deadlines, registration type and approval can therefore be different for each activity.

Event	Activities inside it
TechFest 2026	Hackathon
	Quiz
	Workshop

A standalone activity is also supported. That is an activity that does not belong to an Event.

Activity can have:

• Name, description, type and rules

• Individual / Team / Both registration

• Registration deadline

• Scheduled start and end time

• Location and capacity

• Minimum and maximum team size

• Approval required or not

3. Participant registration

The participant does not simply press "Register" and get accepted. The backend checks the activity rules first.

Examples of checks:

• Is the participant allowed to see and access the event/activity?

• Is registration still open?

• Is the requested registration type allowed?

• For team registration, is the participant a valid team member and leader where required?

• Is the team size within the configured limits?

• Has this participant/team already registered?

• Is capacity still available?

Approval flow

Setting	What happens
Approval not required	Registration becomes APPROVED immediately.
Approval required	Registration starts as PENDING. Organizer can approve or reject it.

Participants can view their registrations and can cancel them according to the implemented rules.

> **Easy explanation:** "The frontend sends the registration request, but the backend decides whether the request satisfies the activity rules."

4. Teams

Teams are used when an activity supports team registration.

The person creating the team becomes the first member and leader. The leader manages members and can transfer leadership.

When a team registers for an activity, the system stores the team members who were part of that registration.

Simple example

Team	Members
ByteForce	Aman (leader)
	Riya
	Karan

For a team activity, the activity rules decide the minimum and maximum team size.

5. College visibility

Event visibility controls who can discover the event.

Visibility	Meaning
COLLEGE_ONLY	Only participants from the organizer's college can see/access it.
OPEN_TO_ALL	Participants from other colleges can also access it, subject to the other rules.

Activities inside an event use the parent event's visibility. Standalone activities can have their own visibility.

This makes the project suitable for both college-only competitions and inter-college events.

6. Tickets and QR codes

A ticket represents permission to access something. The current design has two ticket scopes:

Ticket	Purpose
EVENT	Used for entry to the overall event venue.
ACTIVITY	Used for check-in to a specific activity.

Activity registrations can receive activity QR tickets. For team registrations, tickets can be issued for each member.

The QR payload is checked by the backend before it is accepted. The ticket is not trusted just because the QR exists.

> **Important current limitation:** the code already contains the EVENT ticket and EVENT_ENTRY concepts, but an approved activity registration is not yet fully connected to automatically create/reuse that event-entry ticket. So do not tell the teacher that an activity QR currently guarantees main-gate entry.

7. Event entry vs Activity check-in

This distinction is important for the future design too.

Main gate: check that the person is allowed into the overall event.

Activity room: check that the person is registered for that particular activity.

Place	Check-in type	Ticket
Main event gate	EVENT_ENTRY	EVENT ticket
Hackathon room	ACTIVITY	Hackathon activity ticket
Quiz room	ACTIVITY	Quiz activity ticket

One participant may register for several activities. The intended domain model is one event-entry permission for the event plus separate activity permissions for the activities.

But the automatic connection between those two ticket layers is still a pending backend improvement in the current code.

8. Volunteers and QR scanning

Organizers can invite volunteers either at the event level or for a specific activity. The current volunteer permission is CHECK_IN.

The invitation uses a token and has an expiry period. Accepting an invitation can create a volunteer account when one does not already exist.

Why scope matters:

• An event volunteer can perform event-level check-in.

• An activity volunteer can be limited to that activity.

• The backend checks the assignment before accepting a scan.

This prevents a volunteer assigned to one area from automatically getting access to every check-in operation.

9. Attendance

When a valid QR is scanned, the system records attendance.

The record keeps information such as:

• Which participant checked in

• Which ticket was used

• Which event/activity it belongs to

• Whether it was event entry or activity check-in

• Which volunteer performed the scan

• When the check-in happened

Duplicate scans are handled so the same ticket + check-in type is not recorded as a fresh attendance entry every time.

10. Activity timing

The system stores two kinds of time for activities:

Time	Meaning
Scheduled start/end	What the organizer planned.
Actual start/end	What really happened during execution.

This is useful when an activity starts late. The schedule does not have to be rewritten just because the real activity started later.

The current activity flow supports manual organizer start and complete actions and keeps actual start/end times.

Event and activity lifecycle states used in the model include DRAFT, PUBLISHED, ONGOING and COMPLETED, with CANCELLED also available for events.

11. Login and access control

Users log in and receive authenticated access. The backend uses JWT-based authentication and protects actions based on role.

In simple terms:

• Organizer can use organizer features.

• Participant can use participant features.

• Volunteer can use volunteer check-in features.

This is why the frontend shows different navigation and pages depending on who is logged in.

The backend also checks ownership. For example, an organizer should not be able to manage another organizer's event.

12. What the frontend teammate can say

A good description of her contribution is:

> "I worked on the frontend screens and user flows that connect users to the EventSphere backend. The frontend provides the organizer, participant and volunteer interfaces for the implemented workflows."

Frontend areas represented in the current project include:

- Login / authentication and role-based navigation
- Organizer event and activity management
- Participant event/activity browsing
- Registration and registration history
- Team management
- Organizer approval/rejection screens
- Volunteer management and invitation flow
- Volunteer QR scanner
- Event entry attendance view
- Activity attendance / operations views
- Ticket-related participant screens

13. Quick questions she should be ready for

Question	Easy answer
What is EventSphere?	A platform for managing college events, activities, registrations, tickets, volunteers and attendance.
What is an Activity?	A participant-facing item inside an event, such as a hackathon or quiz.
Why separate Event and Activity?	Because each activity can have its own rules, capacity, deadline and team requirements.
What is a QR ticket?	A digital ticket that the backend verifies before allowing the relevant check-in.
What does a volunteer do?	A volunteer scans tickets for the event/activity scope assigned by the organizer.
Can teams register?	Yes, when the activity allows team registration, with configured size rules.
What if approval is required?	The registration stays PENDING until the organizer reviews it.
What is the current event-entry situation?	The system has separate event-entry ticket/check-in concepts, but activity registration is not yet fully wired to auto-create/reuse the event-entry ticket.

14. What is implemented vs future work

Implemented in the current working project

• Authentication and role-based access

• College-aware visibility

• Event creation, update, publish and lifecycle

• Activity creation and management

• Individual / team / both registration rules

• Registration deadlines and capacity checks

• Team management

• Registration approval/rejection

• Activity tickets and QR verification

• Volunteer invitations with event/activity scope

• Event entry and activity attendance concepts

• Attendance recording and organizer attendance views

• Transaction/concurrency protection for critical operations

Not yet part of the current implemented scope

• Judge assignment and scoring workflow

• Live leaderboard / final results workflow

• Certificate generation

• Participant feedback module

• Analytics/reporting module

• AI-generated event content features

> **Best practice for the presentation:** explain what is actually working today, then describe the other items as planned extensions. This keeps the demo accurate and easy to defend.

Source note: prepared from the current EventSphere implementation reviewed on the working branch **vansh-branch**. This document is intentionally written in simple language for a frontend-focused teammate.